Sistema de Controle de Estoque

Uma plataforma completa, segura e responsiva para gestão de inventário, focada no rastreamento absoluto da vida útil dos produtos, controle financeiro (custo médio ponderado) e geração de relatórios gerenciais.

🚀 Tecnologias Utilizadas

Frontend (web/)

- **Framework:** React 19 com TypeScript
- **Bundler:** Vite
- **Estilização:** Tailwind CSS (v3.4)
- **Componentes UI:** Radix UI (base para Shadcn)
- **Ícones:** Lucide React
- **Roteamento:** React Router Dom
- **Mock de API:** Axios Mock Adapter (para desenvolvimento e testes de UI)

Backend (server/)

- **Runtime:** Node.js (com `tsx` para execução de TypeScript)
- **Framework Web:** Fastify (Alta performance)
- **Validação de Dados:** Zod integrado ao Fastify (`fastify-type-provider-zod`)
- **Upload de Arquivos:** `@fastify/multipart`
- **Banco de Dados:** SQLite (Fácil configuração para desenvolvimento/testes, pronto para escalar para PostgreSQL)
- **ORM:** Prisma
- **Geração de PDF:** Puppeteer (Gera PDFs a partir de templates HTML)

📦 Módulos do Sistema

1. Dashboard (Painel de Controle)

Fornece uma visão panorâmica da saúde do estoque em tempo real.

- **Valor Total Investido:** Cálculo dinâmico de `Custo Médio Ponderado * Saldo Atual`.

- **Giro de Itens:** Somatório físico de todos os itens cadastrados.
- **Alertas Críticos:** Destaque para produtos cujo `currentStock` está igual ou inferior ao `minStock`.
- **Atividade Recente:** Feed estilo "timeline" mostrando as últimas entradas e saídas.

2. Produtos

Gestão do catálogo de itens da empresa.

- **Cadastro:** SKUs únicos, Unidade de Medida (UN, CX, KG), preço de venda e limites de estoque (Mínimo).
- **Custo Médio Automático:** O sistema nunca pede o "Custo" no cadastro, pois ele é recalculado automaticamente a cada nova Entrada (Compra).
- **Listagem:** Tabela com indicadores visuais para itens que precisam de reposição.

3. Movimentações (O Coração do Sistema)

Garante a rastreabilidade (Log de Auditoria) imutável. Nenhuma quantidade de produto muda "magicamente"; tudo exige uma movimentação.

- **Entradas (IN):** Ao registrar uma compra, o estoque do produto aumenta e o Custo Médio é recalculado matematicamente. Permite o anexo do arquivo/imagem da Nota Fiscal (NFe).
- **Saídas (OUT):** Baixa de itens para uso interno, avaria ou venda. Valida se há estoque suficiente antes de prosseguir para evitar saldos negativos.
- **Transações Seguras:** Utiliza `Prisma.$transaction` garantindo que se o registro falhar, o estoque não é alterado (Atomicidade).

4. Relatórios Gerenciais

Geração de documentos oficias em PDF no lado do servidor.

- **Posição de Estoque:** Relatório completo com listagem de produtos, custos, totais e formatação profissional. Utiliza o Puppeteer no backend para renderizar Tailwind HTML diretamente para PDF.

🗄️ Modelagem de Dados (Banco)

A estrutura relacional foca em não perder nenhum rastro de dado financeiro ou físico:

- **Product:** Mantém o saldo consolidado atual (`currentStock`) e o custo real (`averageCost`).
- **Movement:** Log imutável (Data, Tipo IN/OUT, Quantidade, Valor Unitário daquela operação, Responsável).
- **Attachment:** Arquivos de NFe fisicamente salvos no servidor e logicamente vinculados a um Movement.
- **Category, Supplier, Client, User:** Tabelas de apoio cadastradas previamente.

⚙️ Como Executar o Projeto Localmente

O sistema é dividido em duas pastas: `server` (Backend) e `web` (Frontend).

1. Iniciando o Backend

Abra um terminal na raiz do projeto e execute: ```bash cd server npm install

Cria o banco SQLite (dev.db) e roda as migrações/seed

npx prisma db push npx prisma generate npx tsx prisma/seed.ts # (Opcional, caso não use o mock do frontend)

Inicia a API na porta 3333

npm run dev ```

2. Iniciando o Frontend

Em um novo terminal, abra a pasta web: ```bash cd web npm install

Inicia a interface na porta 5173

npm run dev ```

> **Nota sobre o MOCK:** Atualmente, a aplicação web inicializa os dados via `axios-mock-adapter` em `web/src/main.tsx`. Isso permite testar e validar o visual

> imediatamente sem depender do backend. Para conectar o frontend ao backend real, remova a chamada `setupMocks()` no arquivo `main.tsx`.

🔒 Regras de Negócio Importantes

1. **Recálculo de Custo Médio (CM):** `Novo CM = ((Estoque Atual * CM Atual) + (Qtd Nova * Valor Novo)) / (Estoque Atual + Qtd Nova)`
2. **Estoque Negativo:** O Backend lança um erro 400 e reverte a transação se uma "Saída" for maior que o estoque atual disponível.
3. **Imutabilidade:** O Frontend não possui endpoints para "editar" a quantidade de um produto livremente. Somente o módulo de movimentação altera estoques.